
Relatório: Lab 8

Uso Consciente de LLMs e Prompt Engineering

Aluno: Henrique

Data: 13 de Junho de 2026

Parte 1 - Escolha da Ferramenta

- **Nome da ferramenta:** Gemini 3.1Pro
- **Data de uso:** 13/06/2026
- **Versão:** Paga (com acesso à internet)

Parte 2 - Primeira tarefa: explicação conceitual

A Abordagem Universal

Prompt

explique-me o conceito de overflow

Resposta da LLM (Resumo/Trecho)

O conceito de **overflow** (transbordamento) ocorre quando um computador tenta processar ou armazenar um valor que excede a capacidade máxima alocada para aquele tipo de dado na memória... Se você tiver, por exemplo, um número inteiro de 32 bits (`int`), o valor máximo positivo que ele consegue representar é 2.147.483.647. Se você somar 1 a esse número, o computador não terá bits extras... os bits "dão a volta" (*wrap around*), e o número despenca para o menor valor negativo possível (-2.147.483.648).

Roleplay e Especialização

Prompt

Considerando que eu seja um aluno calouro de ciência da computação entusiasta da otimização em detrimento da engenharia de software e você um professor de olimpíadas da computação. Explique-me o conceito de overflow usando exemplos de como usar o overflow para deixar o código em c mais rápido e mais leve sem quebrar por completo o código.

Resposta da LLM (Resumo/Trecho)

O overflow ocorre quando tentamos armazenar um valor maior do que a capacidade de bits daquele tipo de dado. Porém, para usá-lo como arma de otimização sem destruir o seu código, você precisa gravar uma regra de ouro no seu cérebro: **nunca confie no overflow de variáveis com sinal (*signed*)**. ... O segredo mora no **Unsigned Overflow**. O padrão da linguagem C garante que operações com tipos **unsigned** operam estritamente sob aritmética modular.

Comparação: O **Prompt 1** perguntou "o que é", recebendo uma definição de dicionário ilustrada (foco em segurança). O **Prompt 2** estabeleceu um cenário de exceção, resultando em uma explicação pragmática de manipulação de hardware.

Parte 3 - Segunda tarefa: geração de código

Prompt

Assumindo a posição de um professor de olimpíada, faça a versão mais otimizada de leitura de números inteiros com `getchar()` em C para os seus alunos

Código Gerado:

```
1  #include <stdio.h>
2
3  /* Função inline para evitar overhead de chamada de função */
4  inline int read_int(int *n) {
5      int c = getchar_unlocked();
6      if (c == EOF) return 0;
7
8      int sign = 1;
9      *n = 0;
10
11     while (c < '0' || c > '9') {
12         if (c == EOF) return 0;
13         if (c == '-') sign = -1;
14         c = getchar_unlocked();
15     }
16
17     while (c >= '0' && c <= '9') {
18         *n = (*n << 3) + (*n << 1) + (c - '0');
19         c = getchar_unlocked();
20     }
21
22     *n *= sign;
23     return 1;
24 }
25
26 int main() {
27     int valor;
28     while (read_int(&valor)) {
29         printf("%d\n", valor);
30     }
31     return 0;
32 }
```

Explicação do Código: Implementa uma rotina extremamente rápida. Atua no nível do caractere utilizando `getchar_unlocked()` (removendo a trava de *thread-safety*) e aritmética de ponteiros e bits (usando *bitshifts* como `<< 3` e `<< 1` em vez da multiplicação tradicional).

Testes Realizados:

- **Teste 1:** Entrada: 42 -15 0 999 → Saída: 42, -15, 0, 999 (Passou).
- **Teste 2:** Entrada: -845 \n \n 123456789 → Saída: -845, 123456789 (Passou).

Análise Crítica: Excelente para programação competitiva, porém esbarra em problemas de portabilidade (função POSIX, não compila nativamente no MSVC/Windows) e não possui *thread-safety*.

Parte 4 - Melhorando o prompt

Prompt

Assumindo a posição de um professor de olimpíada, faça a versão mais otimizada de leitura de números inteiros em C (linguagem de programação). Restrição de implementação: Não utilize `getchar_unlocked()`, use apenas o `getchar()` padrão para garantir que o código compile nativamente no Windows sem erros. Nível de explicação: Explique a lógica passo a passo com uma didática voltada para calouros de Ciência da Computação. Exemplos: Forneça exemplos de entrada e saída para testarmos o código.

Nova Resposta (Código Seguro):

```
1  #include <stdio.h>
2
3  int read_int_portable(int *n) {
4      int c = getchar();
5      if (c == EOF) return 0;
6
7      int sign = 1;
8      *n = 0;
9
10     while (c < '0' || c > '9') {
11         if (c == EOF) return 0;
12         if (c == '-') sign = -1;
13         c = getchar();
14     }
15
16     while (c >= '0' && c <= '9') {
17         *n = (*n * 10) + (c - '0');
18         c = getchar();
19     }
20
21     *n *= sign;
22     return 1;
23 }
24
25 int main() {
26     int valor;
27     printf("Digite seus numeros: \n");
28     while (read_int_portable(&valor)) {
29         printf("Lido: %d\n", valor);
30     }
31     return 0;
32 }
```

Comparação de Códigos:

Critério	Código 1 (Prompt Original)	Código 2 (Novo Prompt)
Função Base	<code>getchar_unlocked()</code> (POSIX/Linux).	<code>getchar()</code> padrão (Windows/Linux).
Matemática	Deslocamento de bits ($\ll 3$ e $\ll 1$).	Multiplicação decimal tradicional ($\ast 10$).
Adequação	Competições puras (Codeforces, URI).	Software acadêmico / multi-plataforma.

Parte 5 - Verificação crítica

(Análise sobre a primeira versão de código, baseada em `getchar_unlocked()`)

1. **A resposta estava correta?** Sim, no contexto estrito de programação competitiva (máxima otimização possível).
2. **Havia informações incompletas ou incorretas?** Sim, a LLM omitiu que a função pertence ao padrão POSIX e não alertou sobre riscos de *thread-safety* e incompatibilidade com Windows.
3. **Como a resposta foi verificada?** Por meio da inspeção rigorosa do código e simulação de testes práticos com entradas variadas (números, lixo e espaços).
4. **A LLM demonstrou excesso de confiança?** Sim. Ela entregou o código focado inteiramente na velocidade bruta sem apresentar as devidas ressalvas arquiteturais.
5. **Como o prompt poderia ser melhorado?** Adicionando requisitos não funcionais, restrições técnicas e nível de didática exigido (conforme realizado e resolvido na Parte 4).

Parte 6 - Boas práticas de uso

1. **Por que não devemos copiar respostas sem revisão?** Modelos de linguagem podem "alucinar" ou ignorar cenários práticos. Copiar sem revisar significa incorporar vulnerabilidades (como a falha de compatibilidade no Windows da Parte 3) no seu trabalho.
2. **Por que é importante informar os prompts utilizados?** Garante reprodutibilidade e transparência. O prompt reflete o nível de domínio do usuário sobre o problema, mostrando que o raciocínio humano guiou a IA, e não o oposto.
3. **Em quais situações as LLMs ajudam no aprendizado?** Como tutores particulares: explicando conceitos difíceis com analogias, atuando no processo de *debugging* ou resumindo bibliotecas de código.
4. **Em quais situações elas podem atrapalhar?** Quando o aluno terceiriza o esforço cognitivo (gerar a solução final de listas de exercícios) e perde a chance de construir a fundação da lógica de programação.
5. **O que significa utilizar uma LLM como apoio e não como substituta?** Enxergá-la como uma "calculadora avançada de texto". O desenvolvedor mantém o poder de avaliação e veto, usando a IA apenas para automatizar partes braçais do desenvolvimento.

AI Usage Statement

Este trabalho utilizou uma ferramenta de IA generativa para auxiliar na geração de explicações e exemplos de código. Todos os prompts utilizados estão documentados em `prompts.txt`. As respostas foram revisadas, testadas e verificadas pelo autor antes de serem incluídas no relatório.

Reflexão sobre o uso de IA Generativa

A realização deste laboratório proporcionou uma compreensão profunda e prática sobre o papel das ferramentas de Inteligência Artificial Generativa no cotidiano de um estudante de Ciência da Computação. Abaixo, são apresentadas as reflexões fundamentadas nas experiências e nos testes realizados ao longo das tarefas:

1. **O que você aprendeu sobre interação com LLMs?** Aprendi que a interação com uma LLM é um processo iterativo e altamente dependente de contexto. Modelos de linguagem não possuem discernimento próprio ou consciência situacional; eles atuam como espelhos estatísticos do comando fornecido. Quando colocados em um papel específico (como o de um professor de maratona de programação), eles adotam uma persona extrema e executam a instrução de forma literal, o que exige do utilizador um papel ativo de filtro e governança.
2. **Como a qualidade do prompt influenciou os resultados?** A qualidade do prompt foi o fator determinante entre um código perigoso e uma solução robusta de engenharia. No prompt original e ambíguo, a IA sacrificou a portabilidade, a segurança e a clareza didática para atingir a "otimização máxima" através de `getchar_unlocked()`. Quando o prompt foi refinado com restrições explícitas (compatibilidade com Windows, uso de `getchar()` padrão e foco pedagógico para calouros), a resposta mudou drasticamente para um código seguro, multiplataforma e estruturado de forma exemplar.
3. **Em quais situações a ferramenta foi mais útil?** A ferramenta demonstrou um valor extraordinário como tutora conceitual e geradora de estruturas iniciais (*boilerplates*). A capacidade de desmistificar o conceito de *overflow* usando analogias físicas cotidianas (como a garrafa de água) e, logo em seguida, aprofundar-se na aritmética modular de ponteiros de baixo nível prova que as LLMs são excelentes para acelerar a curva de aprendizagem e fornecer diferentes perspectivas sobre o mesmo problema técnico.
4. **Em quais situações foi necessário verificar ou corrigir respostas?** A verificação foi estritamente necessária no momento da avaliação de portabilidade e comportamento do compilador. A IA demonstrou um "excesso de confiança" crítico ao omitir que a otimização com `getchar_unlocked()` quebraria completamente a compilação nativa em ambientes Windows utilizando o MSVC. Isso provou que, em cenários envolvendo APIs de sistemas operacionais, concorrência (*thread-safety*) ou comportamentos indefinidos (*Undefined Behavior* em *signed overflow*), a validação humana e a consulta às documentações oficiais são obrigatórias.
5. **Você utilizaria essa ferramenta em disciplinas futuras? Por quê?** Com certeza utilizarei, mas sob uma nova perspectiva: como uma ferramenta de **apoio ao raciocínio** e nunca como substituta dele. As LLMs funcionam como uma calculadora avançada para engenharia de software; elas eliminam o trabalho mecânico e expandem o repertório de soluções, mas o domínio da lógica, o design do algoritmo e a responsabilidade técnica final devem permanecer sob o controle do estudante. Usar a IA para estudar conceitos e validar abordagens enriquece a formação; usá-la para terceirizar o esforço cognitivo a destrói.